Introduction to Documentation Tools: Doxygen & Google Docs

Dr. Neeta Nain

MNIT Jaipur

September 11, 2025

From LaTeX to Other Documentation Tools

- **Previously:** You have learned the fundamentals of LaTeX for professional-quality typesetting of technical documents.
- **Next:** We will explore two powerful, widely-used tools:
  - **Doxygen** – for automated documentation of code, generating reference manuals directly from source.
  - **Google Docs** – for collaborative, cloud-based editing and rapid document creation.
- Each tool suits different needs: code-centric API docs, team documents, reports, and much more.
- By learning these, you'll be ready for modern documentation workflows in academic, research, and industry settings.

Session Goals

- Understand the importance of technical documentation
- Learn how Doxygen and Google Docs support various documentation needs
- Get hands-on with real-world syntax and collaboration tools
- Compare approaches for structured versus rapid team documentation

Why Documentation?

- Clarifies requirements and specifications
- Enables software maintenance and onboarding
- Supports compliance, auditing, and quality control
- Fosters effective knowledge sharing and collaboration

What is Doxygen?

- Open-source documentation generator for many programming languages (C, C++, Python, Java, etc.)
- Extracts information from special comments in code to produce reference manuals (HTML, LaTeX, PDF)
- Widely used for library and API documentation

Key Features of Doxygen

- Parses source code, extracting function/class/module documentation from comments
- Generates cross-referenced code, call graphs, inheritance diagrams, and more
- Supports Markdown, LaTeX formulas, diagrams (Graphviz), and code examples
- Easy integration with CI/CD workflows

Installing Doxygen

- Windows: Download installer from `https://www.doxygen.nl/download.html`
- macOS: `brew install doxygen`
- Linux: `sudo apt-get install doxygen` (Debian/Ubuntu)
- Check: `doxygen --version`

Basic Doxygen Workflow

1. Write specially formatted comments in your source code
2. Create/Edit a configuration file: `doxygen -g Doxyfile`
3. Run: `doxygen Doxyfile`
4. Browse output: HTML (`html/index.html`) or LaTeX/PDF

Doxygen Comment Syntax (C/C++/Java)

```
/**
 * \brief Brief description of the function.
 *
 * Longer detailed explanation.
 * \param x A parameter description.
 * \return Description of the return value.
 */
int myFunction(int x);
```

Doxygen for Python

```
def foo(bar):
    """
    Brief description.
    More details here.

    :param bar: Bar parameter
    :type bar: int
    :return: Computed value
    :rtype: int
    """
    return bar + 1
```

Common Doxygen Tags

Tag	Purpose
\brief	Short summary of function/class/module
\param	Parameter description
\return	Return value description
\author	Author info
\version	Version
\see	Reference to related items
\note	Additional notes

More Doxygen Example: Documenting Classes

```
/**
 * \class Car
 * \brief Represents a car object.
 *
 * Details about the class.
 */
class Car {
    public:
        /**
         * \brief Start the engine
         * \return True if successful
         */
        bool startEngine();
};
```

Advanced: Grouping and Diagrams

- Use |/| for logical grouping
- Create call graphs, include diagrams (HAVE DOT=YES in Doxyfile)
- Embed diagrams with @dot or plantuml directives
- Add Markdown/HTML tables and images to richen documentation

Doxygen Output Formats

- HTML (default, best for browsing)
- LaTeX/PDF (for printable manuals)
- RTF, XML, Docbook, man pages

```
GENERATE_HTML          = YES
GENERATE_LATEX         = YES
GENERATE_RTF           = NO
GENERATE_XML           = NO
```

Best Practices: Writing for Doxygen

- Express purpose and contract of functions, not just parameters
- Keep brief and detailed descriptions distinct
- Document all public APIs and modules
- Use consistent tag set and layout
- Generate documentation as part of continuous integration

Doxygen with Version Control

- Store configuration (`Doxyfile`) in source repository
- Encourage team to update docs with code reviews
- Publish generated docs to GitHub Pages, internal wiki, etc.

What is Google Docs?

- Cloud-based word processor part of Google Workspace
- Enables live, collaborative editing of documents from any device
- Supports comments, suggestions, version history, templates, and integrations

Key Features of Google Docs

- Real-time multi-user collaboration (see edits as they happen)
- Rich formatting: styles, headings, tables, images, drawing, links, equations
- Comments and suggestions for easy review and feedback
- Version history to track changes and restore previous states
- Export/import: DOCX, PDF, ODT, RTF, HTML, .txt

Creating and Structuring a Document

- Start from a new blank doc or use templates (Reports, Meeting Notes, Manuals, etc.)
- Use headings and styles for consistent organization
- Insert tables, lists, images, footnotes, and equations as needed

Paragraph Styles and Hierarchy

- Title, Subtitle, Heading 1..., Normal Text
- Proper use of styles allows for autogenerated TOC and consistent appearance
- Styles enable accessibility features and better exporting

Collaboration and Sharing

- Click "Share" to invite collaborators via email or link
- Specify view/comment/edit rights
- Add, resolve, assign comments; use "suggesting" for tracked edits
- Integrated chat for live discussions
- Collaborate on the same doc, see live cursors and changes

Version History and Recovery

- Google Docs automatically saves every change
- File $\rightarrow$ Version history to see and restore previous versions
- Name specific versions for key document milestones (submission, approval)

Images, Drawings, and Diagrams

- Insert $\rightarrow$ Image: From upload, camera, web, Drive, or Photos
- Insert $\rightarrow$ Drawing: Built-in tool to create simple diagrams (arrows, shapes)
- Extensions/integrations support lucidchart, diagrams.net, etc.

Tables and Lists

- Insert $\rightarrow$ Table: Organize data, compare options, build schedules
- Use bulleted/numbered lists for structure
- Checklists for action items and collaborative tracking

Equations and Code Snippets

- Insert $\rightarrow$ Equation: Supports basic LaTeX-style math in GUI
- For code: use preformatted / monospace font for inline snippets
- For larger code blocks, use third-party add-ons or paste images

Templates and Formatting Tools

- Use built-in templates for reports, project proposals, etc.
- Customize headers, footers, page numbering
- Use find/replace, spelling/grammar, and suggestive autocorrect

Citations and Bibliography in Google Docs

- Tools $\rightarrow$ Citations: Add sources in MLA, APA, Chicago
- Insert in-text citations and auto-generate a bibliography
- Integrate with citation managers (Zotero, Mendeley, etc.) via add-ons

Exporting and Publishing Documents

- Download as PDF, DOCX, ODT, HTML, etc.
- Publish to web for public view
- Integrate with other Google Workspace apps (Slides, Sheets)

Security, Permissions, and Data Protection

- Flexible document-level permissions
- Set expiration dates and remove access
- Workspace admin controls for organization-wide sharing policies

Integrations with Other Tools

- Seamless integration with Google Drive, Calendar, Meet, Forms
- Connect project management (Trello, Asana), code tools (GitHub), and diagrams via add-ons
- Script automation using Google Apps Script

Best Practices for Google Docs Documentation

- Use a consistent style guide and templates
- Regularly organize content with headings and table of contents
- Assign reviewing/editing roles for major docs
- Clean up resolved comments and suggestions
- Backup versions and export critical milestone deliverables

Doxygen vs. Google Docs

Feature	**Doxygen**	**Google Docs**
Purpose	Code/API Docs	General Docs, Proposals
Collaboration	Indirect (code reviews)	Real-time multi-user
Rich Media	Diagrams in code	Images, tables, drawings
Output	HTML, LaTeX, PDF, etc.	Web, PDF, Word, etc.
Best For	Source code, APIs	Policies, specs, notes

When to Use Which?

- Use **Doxygen** for automatic, up-to-date, code-linked API documentation.
- Use **Google Docs** for collaboration, business proposals, and non-code documents.
- Both together for full project documentation (design in Docs, API in Doxygen).

Try It: Doxygen Mini-Project

1. Download and install Doxygen
2. Comment a small code base (C++ or Python) with Doxygen tags
3. Generate HTML output; open `html/index.html`
4. Explore diagrams and cross-references

Try It: Google Docs Collaboration

1. Create a Google Doc and share with a classmate (edit or suggest mode)
2. Try headings, comments, and suggestions
3. Add a table and embed an image or diagram
4. Review version history and restore a previous version

Key Takeaways

- Doxygen: automates code documentation, integrates with code review
- Google Docs: live collaboration, formatting, sharing, version control
- Documentation quality comes from planning, clarity, and using the right tool

Further Resources

- Doxygen Manual: `https://www.doxygen.nl/manual/`
- Google Docs Help: `https://support.google.com/docs`
- Open-source documentation tools: Sphinx, MkDocs, ReadTheDocs
- Book: "Docs for Developers" by Fogarty et al.

Thank you!!!!